

Plug-and-Play LLMs: Designing a Modular Unity System for Real-Time Multi-Model Interaction

Efe Oral - Matrikel number: 2965307 - efeoral@gmail.com

HCI Praktikum (Examination: 340151) Report supervised by Dr. David Obremski

Abstract

This project presents a modular Unity-based system that allows users to interact with various locally hosted large language models (LLMs) in real-time. The goal was to build a flexible and immersive environment where users can type or speak prompts and receive streamed, context-aware responses through both on-screen text and speech. By using open-source models hosted via Ollama, the system runs offline (for typed input), helping reduce cost and improve privacy, especially for sensitive use cases like therapy. Speech input is handled with Microsoft Azure, and speech output is powered by ElevenLabs. The Unity interface features an animated virtual character with lip-sync, creating a more natural interaction. The system supports easy switching between multiple LLMs, and new models can be integrated with minimal setup. Thanks to its plug-and-play structure and real-time performance, this project opens up possibilities in areas like therapy, education, game development, and research.

1. INTRODUCTION

This project is an extension of a previous one by David Obremski, created as part of the computer science praktikum. The main goal is to build a system where users can interact with different large language models (LLMs) in real-time through a Unity-based interface. Users can communicate with the models by typing Fig. 1 or speaking, and the system responds by showing the answer on the screen and reading it out loud using Text-to-Speech.

The LLMs are connected using Ollama, which makes it possible to run models locally without needing an internet connection. This not only improves accessibility but also gives more control over privacy. Right now, the system supports several open-source LLMs as can be seen in table I.

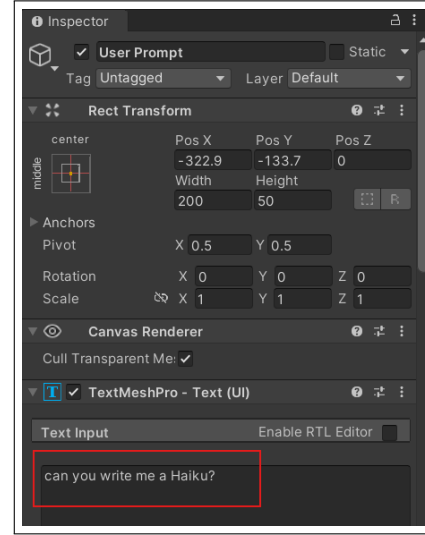


Fig. 1: Typed Text Input

However, new models can be added easily, thanks to the modular and plug and play structure of the system. These were just the LLMs that are popular and also diverse in their own sense.

Model Name	Size (Parameters)
Llama3	8B
Gemma	2B
DeepSeek	1.5B
Phi3	3.8B
Qwen2.5	0.5B
Granite3	1B

Table I. : Currently supported LLMs in the system

Unity was chosen because it allows the use of an animated virtual character with facial expressions and lip-sync. This adds an extra layer of immersion. There are studies (Lim, Schmäzle, and Bente, 2024) showing that when AI has a virtual body (something people can see and relate to), users are more likely to interact with (Gall et al., 2021) it and feel comfortable. That is especially useful for scenarios like

personal therapy, where building trust and engagement is really important. One of the bigger goals of the project is to explore the idea of a personal AI therapy assistant, a virtual character that listens, responds, and interacts in a more human-like way. This kind of system could also be helpful for learning environments, games, or just general AI research.

2. MOTIVATION

This project was built as an alternative to using popular LLMs like ChatGPT. While ChatGPT is powerful and widely used, it comes with a few limitations. First of all, it is not open source, and the paid version can be expensive for regular use. It also requires an internet connection, which means you are constantly sending data to external servers, and that raises some privacy concerns, especially if the application involves sensitive topics like therapy or personal conversations. That is where open-source models that run locally, like those offered by Ollama, come in. With Ollama, all the LLMs run directly on the user's machine, so there is no need to send any prompts or responses to external servers*. This can be really important for things like therapy, where confidentiality and data security are a big deal. Running everything locally means the user has full control over their own data, nothing is stored or processed in the cloud, and no third parties are involved. This is also useful for educational settings where user data might need to be protected, or for developers working on AI tools in sensitive domains. We do not want personal conversations to leave the user's device unless absolutely necessary. Another limitation with services like ChatGPT is the lack of control. You do not get to choose exactly which model you use, and you can not easily customize or switch between them. In this project, I wanted to create a system where users can not only pick from different LLMs based on their needs (e.g., size, speed, tone), but also easily swap them in real-time while interacting. Another key motivation was long-term reliability and control. When using local, open-source models, you are not dependent on the policies or pricing of a third-party provider. If a certain model gets discontinued or updated in an unwanted way, you can just keep using the version that works for you. This level of control ensures the system can stay functional and stable over time, even as external services or APIs change. Speed and responsiveness were also major goals. When users are talking to a virtual character, even small delays can break the immersion. To improve

the interaction flow, the system streams the LLM's response to Unity sentence by sentence, instead of waiting for the entire reply to be generated. This allows the virtual agent to respond faster and makes the whole experience feel much more dynamic and real-time. Also, not everyone has access to high-end hardware or fast internet all the time. For that, the possibility of running smaller LLMs locally gives the option to scale down for older systems, making the system more accessible and flexible for different environments.

So in summary, the main motivations behind the project were:

- (1) Reducing cost by avoiding paid APIs
- (2) Running everything offline to improve privacy and security*
- (3) Having more control over which LLMs are used
- (4) Supporting near-real-time interaction through streaming
- (5) Offering flexibility with different model sizes and speeds
- (6) Gaining long-term reliability by using open-source models locally

* This project does **not** need internet connection when **typing** the prompt. However, to be able to use Speech-to-text and Text-to-Speech feature, internet connection is currently needed.

3. RELATED WORKS

Embodied AI combines artificial intelligence with a physical or virtual presence, enabling systems to interact with users in more natural and engaging ways. Several projects have explored this intersection, each with varying focuses and capabilities. Let us look at some studies to understand the motivation for embodied AI and what the trend is.

Arthur: Memory-Enhanced Embodied Conversational Agent (Knob et al., 2021)

Arthur is an embodied conversational agent developed to support more personalized and emotionally intelligent communication. The system integrates multiple AI capabilities, including speech recognition, facial emotion detection, and memory-based di-

dialogue management. Arthur can recognize individual users and track information about them across conversations, such as names, preferences, and previously discussed topics. This memory is stored in a structured format and retrieved dynamically to maintain context and personalize future responses. In addition to memory, Arthur incorporates basic emotion recognition using facial expression analysis. When a user interacts with Arthur through a camera interface, the system estimates their emotional state and adjusts its responses accordingly, modifying tone, empathy level, or the complexity of its language.

[Side Note] As a future work 7, recognizing emotion through text input and generating an appropriate emotion for the response by analyzing the input and using that emotion to play different types of animations, facial expressions and even changing the tone of the voice can be explored.

The system supports both text and voice interaction, creating a multimodal user experience. However, Arthur is designed to run primarily through online services and cloud APIs, meaning it does not offer full offline capability or the ability to host multiple LLMs locally. In contrast, our system focuses on modularity, privacy, and offline functionality, making it more adaptable for different environments and use cases like virtual therapy or education.

Dobby: GPT-4 Powered Service Robot (Stark et al., 2024)

Dobby is a research prototype that combines a GPT-4-based language model with a mobile robot platform to explore real world human-AI interaction. The system is designed to perform basic service tasks such as fetching objects, answering user queries, or assisting in daily routines, all through natural language dialogue. The core of Dobby’s architecture connects a large-scale LLM (via the OpenAI GPT-4 API) with a Robot Operating System-based system. When a user gives a voice command, the input is transcribed using a speech recognition module, passed to GPT-4 for interpretation, and then mapped to high-level robotic actions. These actions are executed by the robot’s onboard control system, enabling it to move, identify objects using its vision sensors, or respond conversationally. One key feature of Dobby is its ability to translate abstract user instructions into specific physical behaviors. For example, a user could say “Bring me something to drink,” and the model would infer a plan involving navigation, object detection, and manipulation. However, all language generation and

action decision are performed in the cloud via GPT-4, making the system heavily dependent on a reliable internet connection. Additionally, Dobby’s design is tightly coupled with its hardware and specific task environment. It is not designed for general deployment, and it does not support local model hosting or modular customization of the LLM backend. In contrast, our system emphasizes flexibility, privacy, and hardware-independence, allowing users to switch between LLMs and input methods in real-time, all while running locally without relying on internet-based APIs.

STARie: Peer-like ECA for Children’s Storytelling (Li and Xu, 2023)

STARie is an embodied conversational agent developed to support storytelling activities for children. The system presents a youngling virtual character that engages with young users by asking questions, reacting to their stories, and providing narrative prompts. It uses GPT-3 for generating dialogue and natural language responses, which are then animated using a talking head model that produces realistic facial animations from speech input. The design of STARie emphasizes ethical considerations for child interaction, such as limiting potentially harmful language and ensuring a friendly, age-appropriate tone. While STARie shows how embodiment and natural language generation can be combined effectively for educational purposes, it operates entirely through cloud-based services, meaning it depends on an internet connection and external APIs for both text generation and animation. Additionally, the system does not offer modular support for experimenting with different LLMs, nor does it provide local execution capabilities. In contrast, our system allows for full control over model selection and runs LLMs locally, making it more flexible, privacy-friendly, and adaptable to a variety of use cases beyond storytelling.

Embodied AI: Enhancing Human-AI Interaction (Pfeifer and Iida, 2004)

The idea of embodiment in artificial intelligence has shifted how we think about building intelligent systems. Instead of seeing AI as just a tool that processes inputs and gives outputs, embodiment focuses on giving AI a physical (or at least visual) presence that can interact with its environment, also with people, in a more natural and intuitive way. In the paper “Embodied Artificial Intelligence: Trends and Challenges”(Pfeifer and Iida, 2004), Pfeifer and Iida argue that intelligence is not just about computation.

It also depends on how an agent is physically situated in the world, how it senses, moves, and reacts to its surroundings. In other words, being embodied allows an AI system to learn and behave in ways that are grounded in real experience, not just abstract data. This makes the interaction feel more adaptive, human-like, and meaningful.

Our system follows this same philosophy by combining a virtual agent (with animations, gestures and lip-sync) and a local language model that responds to both voice and text inputs in real-time. Instead of being just a chatbot in a window, it is more life-like, giving the user a stronger sense of presence. That embodiment helps make conversations feel more personal and engaging, especially for use cases like therapy or education. At the same time, we designed the system to stay private and customizable. By running models locally, we keep user data secure and avoid relying on external APIs. And since everything is modular, developers can swap in different LLMs or input/output methods based on their needs. In that sense, this project brings together the practical benefits of embodiment with the flexibility of modern AI tools.

Comparison with Our System

The shared factor among studies discussed here is that embodiment in AI leads to deeper user engagement, increased trust, and more emotionally effective interactions, especially in areas like therapy, education, and support systems. When users interact with a virtual character that speaks, reacts, or moves like a human, they're more likely to feel connected and open up, even if they know it's AI. Embodiment also enhances memory retention, helps reduce feelings of isolation, and creates a stronger sense of presence. These findings suggest that embodiment is not just a visual trick, it can meaningfully enhance the effectiveness of AI systems, especially in sensitive or emotionally charged environments.

Because of these benefits, we chose to develop our system around an embodied AI agent. What makes our approach unique is the combination of embodiment with new features like real-time sentence streaming, modular support for multiple LLMs, and dual input options (voice and text). While previous systems often focused on either the conversational model or the animation side, we aimed to merge both into a unified and accessible system that can run locally, adapt to user needs, and be easily extended in the future.

4. METHODOLOGY

4.1 System Overview

The system is built in Unity and designed to be modular and flexible, allowing users to interact with different LLMs in real-time. It supports both text and speech input, and the responses are delivered through a combination of on-screen text and Text-to-Speech (TTS) audio. The focus was on making the system responsive, immersive, and easy to extend or modify based on future needs or different applications.

4.2 LLM Integration via Ollama

Ollama is used as the backend for hosting open-source LLMs locally. It allows user to pick and run models like Llama3, Gemma, Phi3, and others directly on the user's machine, without needing an internet connection. Unity sends the user's prompt to the Ollama server through a `UnityWebRequest`, formatted as a `JSON POST` request. The LLM response is then streamed back sentence by sentence, like a typewriter effect, which is key to making the interaction feel faster and more natural.

The JSON payload sent from Unity to the Ollama API includes the following fields:

- `"model"`: the name of the selected model (e.g., `"llama3"`)
- `"prompt"`: the user's input
- `"stream"`: set to `true` to enable streaming responses

An example JSON request looks like this:

```
{
  "model": "llama3",
  "prompt": "Hello, how are you?",
  "stream": true
}
```

4.3 Real-Time Streaming and Display

To make the interaction feel fast and natural, the system handles the LLM responses as a live stream rather than waiting for the full response to be generated. This is possible thanks to Ollama's support for streaming as we see that `stream` can be turned on and off in the JSON request, which sends back the generated text piece by piece as the model produces it. On the Unity side, the streamed response is handled using a coroutine that listens for incoming data in real-time. Each incoming fragment (chunk of the response) contains a portion of the full response, usually a sentence or part of a sentence, wrapped

in a JSON object. The coroutine reads these fragments line-by-line using a buffered stream, parses the JSON, and extracts the "response" field. As each piece of text arrives, it's immediately displayed in the Unity UI using a typewriter-style effect. This gives the impression that the virtual character is typing as it thinks, and it helps reduce the perceived latency between input and output. Behind the scenes, the text is also saved sentence-by-sentence so that it can be passed directly to the text-to-speech system. There's also a built-in interruption mechanism. If the user presses the "Stop" button, a boolean flag is flipped, which causes the coroutine to halt immediately, both for the text display and the speech output. This is useful when the user wants to cut off a long or unexpected response and keeps the interaction flexible. The result is a responsive system that feels close to real-time even when working with fairly large models. Because responses are streamed in small chunks and displayed as soon as they arrive, the delay between sending a prompt and seeing feedback is kept to a minimum.

4.4 Speech-to-Text (STT)

To allow users to interact with the system using their voice, we integrated a STT pipeline using Microsoft Azure Cognitive Services. When the user presses the microphone button in the Unity interface, the system begins recording the audio. The captured audio is stored in a .WAV format, which is then sent as a stream to the Azure STT API. The request includes:

- (1) The audio stream (WAV format)
- (2) A subscription key for authentication
- (3) Language and region settings (e.g., "en-US")

Azure processes the audio and returns the transcribed text, which is parsed and immediately fed into the prompt pipeline, exactly like a typed input. From that point forward, the conversation continues as if the user had typed the prompt manually.

We used asynchronous handling to make sure the UI does not freeze while waiting for the speech result. Once the transcription is received, the system updates the prompt field and triggers the LLM request with the recognized sentence. Here's a simplified version of the flow:

- (1) **User speaks** → Unity captures audio
- (2) **Audio is sent to Azure** → returns transcript
- (3) **Transcript becomes prompt** → sent to LLM
- (4) **Response is streamed back** → shown and spoken

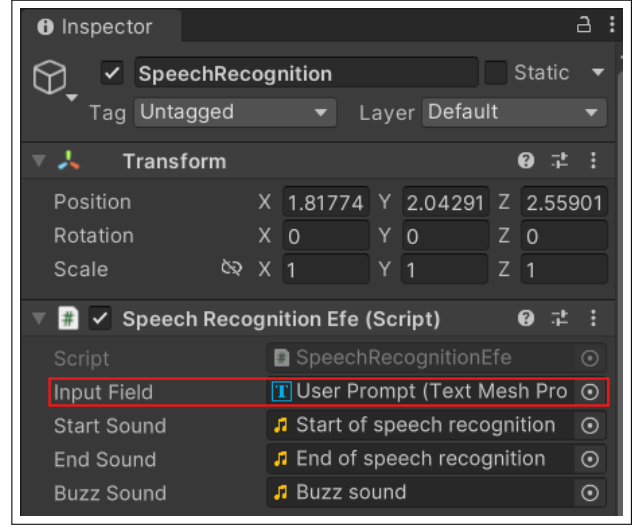


Fig. 2: Recognized Speech Passed to User Prompt

Including speech as an input method is more than just a convenience, it also adds to the immersive quality of the system. A recent study (Konenkov et al., 2024) found that combining speech-to-text and text-to-speech in virtual environments led to increased user comfort, engagement, and task efficiency compared to traditional interaction methods. The study shows that users tend to feel more. Currently, internet access is required for speech-to-text to work, since Azure is a cloud service. One potential improvement for the future would be to replace this with an offline or open-source STT model to make the system fully self-contained.

4.5 Text-to-Speech (TTS)

After the full response from the LLM is received and displayed sentence by sentence in the Unity interface, the complete message is passed to the ElevenLabs Text-to-Speech API to generate speech audio. One needs to use their personal API key. This can be done by pasting the personal API key to "Api Key" section in figure 3. Unlike the streaming display, the TTS system processes the full response as a single unit and returns a complete audio clip. The integration uses the official ElevenLabs C# SDK, which connects to the ElevenLabs server using an API key. A voice is selected from the available voice list (retrieved using `VoicesEndpoint.GetAllVoicesAsync()`), and the full response text is passed along with default voice settings to `TextToSpeechEndpoint.TextToSpeechAsync()` function. A study by (Cabral et al., 2017) explored how synthetic voices affect users' perceptions of virtual

characters in terms of expressiveness, likeability, and clarity. Their findings showed that even though human voices were perceived as more natural and expressive, synthetic voices still played a key role in shaping how users evaluated the character’s communication. This supports our use of high-quality TTS for creating a more engaging and believable interaction, even when a fully human-like voice isn’t available. This supports the use of realistic voice synthesis in our system, where ElevenLabs provides natural-sounding speech that enhances the overall experience, especially when combined with lip-sync and visual feedback. Going back to our pipeline, the retrieved audio is returned as a .mp3 file, which is downloaded to the PC and converted into a Unity AudioClip. Once loaded, the audio is automatically assigned to the virtual character’s AudioSource component and played back in the scene Fig. 3.

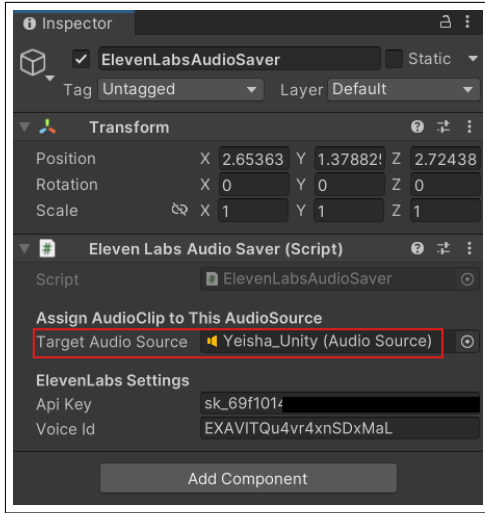


Fig. 3: Assigning the .mp3 File to AI Agent Yeisha

At this point, lip-sync animations are also triggered to create a more immersive and lifelike effect during playback. Although the LLM’s text response is streamed and shown sentence by sentence, the audio is generated after the full text is available. This simplifies the timing between visuals and audio, since both are triggered only after the entire message is ready. However, it does create a slight delay before the virtual agent starts speaking, especially for longer responses. If the user presses the “Stop” button during playback, the system immediately halts the AudioSource and clears any queued animations. This helps the system feel more responsive, giving the user control over when to interrupt the agent. The current design uses ElevenLabs’ cloud API, which requires an

internet connection. As a potential improvement [7], a local TTS solution could be integrated in the future. Another interesting upgrade would be to stream audio sentence by sentence from ElevenLabs in sync with the typewriter-style text, further enhancing the sense of real-time feedback.

4.6 Conversation Handling

One of the key features of the system is its ability to support context-aware conversations, where the AI does not just respond to single, isolated prompts, but remembers what has been said previously. This gives the interaction a much more natural and flowing feel, especially during back-and-forth dialogue. To achieve this, the system stores the full chat history, including both user prompts and LLM responses, in memory during a session. Every time the user sends a new prompt, the system sends both the new message and the full conversation history to the LLM as part of the input payload. This allows the model to see the current message in context and generate a more relevant and coherent reply. What makes this system stand out is that this same chat history can be used **across different LLMs** in real time. At any point, the user can switch to another model (for example, from LLaMA3 to DeepSeek), and the new model will receive the same conversation history, as if it had been part of the entire dialogue from the start Fig. 4.

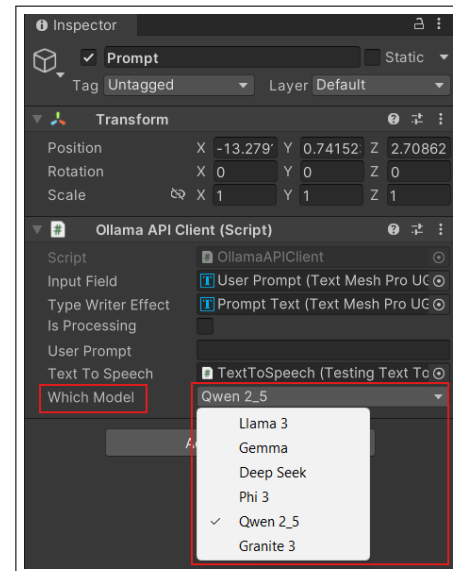


Fig. 4: Choosing Different LLMs Dynamically

This creates a seamless, flexible experience where users can explore how different models handle the

same conversation without losing continuity. Since all the models are accessed through Ollama using the same prompt format and API structure, the system can send the context history along with the prompt no matter which model is selected. No special adaptation is required for each LLM. This makes it very easy to compare models mid-session, or to switch to a faster or more accurate model depending on the user’s goals. To keep things efficient, the stored history is limited to a certain number of past exchanges, enough to give useful context, but not so much that it overwhelms the model’s context window or slows down performance. With trial and error this history messages is set to 5, but it can be adjusted by the user. Also, if a response is cut off (for example, when the user presses “Stop”), it is not saved, ensuring that only complete and relevant information is carried forward. This ability to share and reuse context across models on the fly is one of the most unique aspects and contribution of the system. It turns what is usually a single-model interaction into a flexible, multi-model dialogue that can evolve dynamically as the user explores different options.

A recent study (Zhang et al., 2024) explored a similar concept in a multi-agent framework, where different LLMs collaborate on long-context tasks by sharing and processing the same context chain. Their work shows that distributing context across multiple models can lead to more coherent and accurate results, especially for complex or multiple interactions. While their system focuses on collaboration between LLMs in sequence, our system applies a related idea to real-time conversations, allowing the user to switch between models on the fly while still preserving the full conversation history. This flexible, model-agnostic approach to context sharing makes it easier to test, compare, and personalize interactions with different LLMs.

4.7 Applications

This system offers versatile applications across domains where real-time, immersive interaction with AI is beneficial. One of the most promising use case is in virtual therapy and mental health support. By enabling users to engage with an embodied AI character through voice or text in a private, offline setting, the system ensures data privacy and user control. This approach aligns with findings from a recent study published by Zisquit et al., 2025, which demonstrated the efficacy of AI-enhanced virtual reality self-talk in psychological counseling, highlighting the advantages

of integrating AI agents into self-guided therapy sessions.

When it comes to education, the system can function as an AI teaching assistant, providing real-time answers and guidance to students. Research published in Educational Technology Journal (Labadze, Grigolia, and Machaidze, 2023) emphasizes the benefits of AI-powered teaching assistants in enhancing student learning experiences, offering personalized support, and improving engagement. The system’s capability to switch between different LLMs on the fly allows for adaptability to various educational needs and preferences.

Beyond that, the project is a great fit for prototyping game characters, role-playing experiences, or research into human–AI interaction. Since the core structure is modular, developers can experiment with different LLM behaviors, voices, or visual agents without changing the whole system.

For a full visual overview pipeline of the system, see the diagrams; general overview diagram *Fig: 6* and detailed overview diagram *Fig: 7*, included in the Appendix section [7].

Limitations

While the system is designed to enable real-time interaction, there are still a few technical limitations and performance-related trade-offs to consider. The first major limitation is latency, which can vary depending on the size of the selected LLM and the hardware it’s running on. For example, larger models (typically those above 7B parameters, such as LLaMA3 8B) tend to introduce longer delays before generating the first sentence, especially on systems with limited CPU or GPU resources. Although responses are streamed and displayed incrementally, there is still a delay before the typewriter effect begins.

Another trade-off exists between response speed and output quality. Smaller models like DeepSeek (1.5B) or Qwen2.5 (0.5B) respond much faster and are ideal for low-spec machines or time-sensitive interactions. However, these smaller models may give more generic or shallow answers. Larger models, while slower, tend to provide more coherent, complex replies, but this comes at the cost of higher memory and compute usage.

A second limitation is the reliance on cloud-based services for voice input and output. While the language model itself runs locally, both Microsoft Azure (Speech-to-Text) and ElevenLabs (Text-to-Speech) require an internet connection. This means full of

fine interaction is only possible when using text input/output only. In sensitive scenarios, such as therapy sessions or private conversations, this dependency may raise privacy concerns, since user audio is sent to third-party APIs. As a future improvement, replacing these cloud-based modules with fully local speech recognition and synthesis models would not only improve privacy but also remove the need for internet access entirely, making the system more robust and self-contained.

5. DEVELOPER NOTES & GUIDANCE

If someone wants to build a similar system or expand on this project, there are a few things worth knowing ahead of time, both challenges and tips that can make development smoother. If you have any questions or recommendations, reach me out at my email: efeoral@gmail.com

5.1 Model Size vs. Performance

One of the first challenges is dealing with different LLM sizes. While larger models (like Llama3 8B) give better results, they also slow down generation and use a lot more memory. For developers working on older systems or laptops, it's best to start with lighter models (like Qwen2.5 or Phi3 mini) and only scale up once the system is stable. This makes testing much easier and reduces crashes or lag during streaming.

5.2 Streaming and Responsiveness

Real-time streaming from Ollama works well, but some models respond with slightly different chunk sizes or timing. A good trick is to buffer sentence ends (e.g., periods or question marks) before pushing them to TTS. This avoids cutting off speech too early or sending incomplete audio. Also, using async coroutines in Unity is key to keeping the UI responsive.

5.3 Voice and Audio Integration

Using ElevenLabs and Azure is straightforward, but both require an internet connection. You just need to add your personal API key in the "Api Key" section in figure 3. Also, one need to import ElevenLabs package to Unity via Packet manager Fig: 5 from this GitHub repository [RageAgainstThePixel](#). A side note about ElevenLabs: I started the project with a free account, and when my credit limit was done I could not use TTS and ElevenLabs does not send an error about this. That is why the program would not work. I thought there was a problem elsewhere in the project so I lost a lot of time searching until

I found out it was about my ElevenLabs credit, so I switched to a paid version. If the goal is full offline capability, developers should look into open-source options like Whisper (for STT) and Coqui or Piper (for TTS). These take more setup and hardware power but offer total privacy and control. Also, make sure to handle stopping of audio playback properly, Unity's `AudioSource.Stop()` is sometimes not enough to fully cancel playback if something is queued.

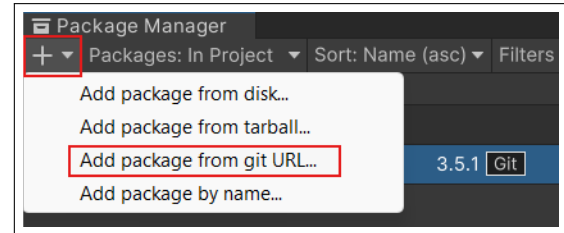


Fig. 5: Adding RageAgainstThePixel Package

5.4 Unity Web Requests and JSON Handling

When dealing with Ollama, always make sure your JSON requests and responses are properly structured. Proper structure of JSON request can be found at 4.2. `UnityWebRequest` does not show detailed errors by default, so use logging or a try-catch block to debug malformed JSON or network issues. Also, keep an eye on the prompt history size, too much history can break small models or slow down generation. Default size is set to 5 which is tested to be the sweet spot between context history and performance.

5.5 Modular Design

This project is highly modular, and that helped a lot during testing. If you are extending the system, keep things modular wherever possible: have separate scripts for prompt sending, speech processing, and character animation. This way, it is easier to swap out or debug individual components without breaking everything.

5.6 Testing LLMs

Different models sometimes behave very differently to the same prompt. For best results, it helps to test each model with a wide range of prompts (short, long, vague, and specific). Logging their responses in separate files can help you find which one fits your application best. Also, not all models may support the same prompt formatting, so keeping your prompts simple and clean helps with compatibility.

6. CONCLUSION

This project introduced a modular Unity-based system for real-time interaction with multiple large language models, focused on creating a more human-like and immersive experience through the combination of voice, text, and AI embodiment. By leveraging open-source LLMs through Ollama, the system runs locally and offline (for text-based interaction), which not only improves privacy and data control but also reduces reliance on cloud APIs and paid services. A key feature of the system is its ability to support streamed responses, giving users faster, more responsive feedback that feels more natural, especially when paired with Text-to-Speech and animated lip-sync in the virtual character. The dual input method (voice and text) makes the system flexible for different users and scenarios, whether it's education, mental health support, or prototyping interactive agents. What sets this project apart from similar works is its **plug-and-play modularity**, full local model hosting, and the unique **ability to preserve context across multiple LLMs in real time**. This allows users to switch models mid-conversation without losing the flow, which opens up new possibilities for comparing LLMs or tailoring behavior on the fly. Ultimately, this project demonstrates how local LLMs, real-time streaming, voice integration, and virtual embodiment can work together to build more immersive and accessible AI experiences, all while keeping performance and privacy in mind. It is a step toward more personalized, human-centered AI systems that can adapt to different users and grow over time.

7. FUTURE WORK

There are a few directions this project could take in the future. Small improvements like allowing the user to choose the voice for Text-to-Speech or allowing for multiple language inputs (e.g. German). Furthermore, one of the biggest improvements would be replacing the cloud-based speech services with local alternatives. Right now, the system uses ElevenLabs for Text-to-Speech and Microsoft Azure for Speech-to-Text, both of which require an internet connection and raise some privacy concerns. For use cases like therapy or other sensitive conversations, it would be much better to run everything locally to keep user data private and secure. Integrating an open-source, offline TTS and STT solution would make the entire pipeline fully local and self-contained. Another potential improvement is optimizing the system for better performance. Depending on the size of the LLM being used, there can be noticeable delays, es-

pecially when using larger models. In the future, it would be useful to explore lightweight model architectures, better GPU scheduling, or techniques to reduce latency without losing too much quality. Another interesting direction would be to enhance the behavior of the virtual character agent itself. Adding more animations, facial expressions, or even gesture-based responses could help make the interaction feel more lifelike and personalized. Over time, the system could develop into a more intelligent and emotionally aware assistant, not only for therapy, but also for areas like education, training, or personal use.

APPENDIX

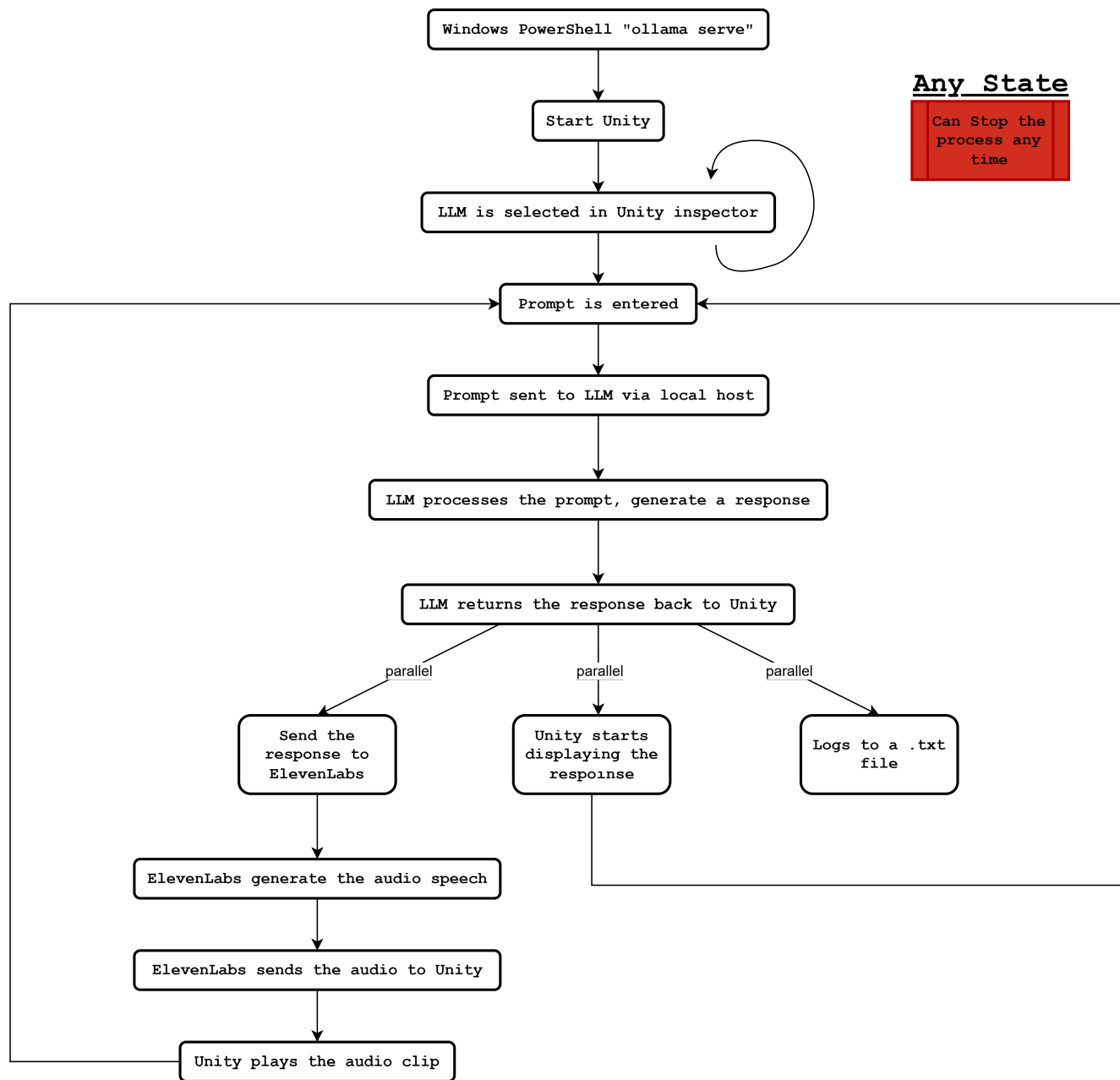
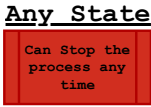


Fig. 6: Program Cycle - General Overview



11

References

- Cabral, J. P., Cowan, B. R., Zibrek, K., & McDonnell, R. (2017). The influence of synthetic voice on the evaluation of a virtual character. *Interspeech*, 229–233 (cit. on p. 5).
- Gall, D., Roth, D., Stauffert, J.-P., Zarges, J., & Latoschik, M. E. (2021). Embodiment in virtual reality intensifies emotional responses to virtual stimuli. *Frontiers in Psychology*, 12, 674179 (cit. on p. 1).
- Knob, P., Dias, W. S., Kuniechick, N., Moraes, J., & Musse, S. R. (2021). Arthur: A new eca that uses memory to improve communication. *2021 IEEE 15th International Conference on Semantic Computing (ICSC)*, 163–170 (cit. on p. 2).
- Konenkov, M., Lykov, A., Trinitatova, D., & Tsetserukou, D. (2024). Vr-gpt: Visual language model for intelligent virtual reality applications. *arXiv preprint arXiv:2405.11537* (cit. on p. 5).
- Labadze, L., Grigolia, M., & Machaidze, L. (2023). Role of ai chatbots in education: Systematic literature review. *International Journal of Educational Technology in Higher Education*, 20(1), 56 (cit. on p. 7).
- Li, Z., & Xu, Y. (2023). Designing a realistic peer-like embodied conversational agent for supporting children’s storytelling. *arXiv preprint arXiv:2304.09399* (cit. on p. 3).
- Lim, S., Schmälzle, R., & Bente, G. (2024). Artificial social influence via human-embodied ai agent interaction in immersive virtual reality (vr): Effects of similarity-matching during health conversations. *arXiv preprint arXiv:2406.05486* (cit. on p. 1).
- Pfeifer, R., & Iida, F. (2004). Embodied artificial intelligence: Trends and challenges. *Lecture notes in computer science*, 1–26 (cit. on p. 3).
- Stark, C., Chun, B., Charleston, C., Ravi, V., Pabon, L., Sunkari, S., Mohan, T., Stone, P., & Hart, J. (2024). Dobby: A conversational service robot driven by gpt-4. *2024 33rd IEEE International Conference on Robot and Human Interactive Communication (ROMAN)*, 1362–1369 (cit. on p. 3).
- Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., & Arik, S. (2024). Chain of agents: Large language models collaborating on long-context tasks. *Advances in Neural Information Processing Systems*, 37, 132208–132237 (cit. on p. 7).
- Zisquit, M., Shoa, A., Oliva, R., Perry, S., Spanlang, B., Klomek, A. B., Slater, M., Friedman, D., et al. (2025). Ai-enhanced virtual reality self-talk for psychological counseling: Formative qualitative study. *JMIR Formative Research*, 9(1), e67782 (cit. on p. 7).